CS3423: Compilers I — Assignment 2

Department of Computer Science and Engineering, IIT Hyderabad

Release: April 2026 Total Marks: 75
Deadline: 25th April, 2026 Mode: Individual

This assignment develops practical familiarity with the LLVM compiler infrastructure. You will generate LLVM IR from C/C++ programs and carefully analyse the output. By completing it you should be able to:

- Install and operate the LLVM/Clang 17 toolchain.
- Translate specific C/C++ programs to LLVM IR and read the output.
- Explain how control flow, data structures, and floating-point operations are represented in LLVM IR.
- Understand the LLVM Compiler optimizations.

Environment Setup

All questions require Clang/LLVM **17.x**. Choose one option below and verify your installation before attempting any question.

Option A — Prebuilt Binaries (Recommended)

1. Download the binary release for your OS from:
   `https://github.com/llvm/llvm-project/releases/tag/llvmorg-17.0.6`
2. Extract and add the `bin/` directory to your `PATH`:

```
tar -xf LLVM-17.0.6-*.tar.xz
export PATH=$PATH:/path/to/llvm-17.0.6/bin
```

3. Verify both tools are available:

```
clang --version   # should print: clang version 17.x.x
llc   --version   # should print: LLVM version 17.x.x
```

Option B — Build from Source

1. Clone the `release/17.x` branch:

```
git clone --depth 1 --branch release/17.x \
    https://github.com/llvm/llvm-project.git
```

2. Follow the official build guide at `https://llvm.org/docs/GettingStarted.html`.

Note: The source build is resource-intensive. Attempt it only if your machine has at least 30 GB of free disk space and 16 GB of RAM. If you are unsure, use Option A.

Generating IR — Quick Reference

Use `-O0` (optimisations disabled) for all questions unless explicitly instructed otherwise:

```
clang -S -emit-llvm -O0 -o output.ll input.c
```

Assignment Questions

Q1 LLVM IR Generation and Core Constructs [25 marks]

Write each of the four C programs below, generate its IR, and answer the corresponding analysis question. Include both the C source and the relevant IR snippet in your report.

Marks: (a) 5 (b) 8 (c) 7 (d) 5

(a) **Local and global variables.** Write a program declaring (i) a global integer, (ii) a local integer, and (iii) a local integer initialised with `a + b * c`. For each variable, identify the corresponding LLVM instruction (`alloca`, `load`, `store`, or `@global`). Explain in one paragraph why local variables appear as `alloca` instructions rather than as named registers.

(b) **Conditionals and Loops.** Write C programs for each of the following constructs, generate the IR, and briefly describe how each is represented: `if-else`, `switch`, `for`, `while`, and `do-while`. For the loop variants, rewrite the same loop in all three forms and compare the IR. Are `for` and `while` identical? Where does the condition check appear in the `do-while` IR compared to the other two?

(c) **Function calls and the call stack.** Write a `main` that calls a helper `int square(int x)`. Locate and explain: (i) the `define` and `declare` keywords, (ii) how arguments are passed by value and the return value is conveyed back, and (iii) the role of the `ret` instruction.

(d) **Type sizes and casting.** Write a program that assigns a `double` to an `int` (narrowing) and an `int` to a `long` (widening). Identify the LLVM cast instructions used (`fptosi`, `sext`, `zext`, `trunc`) and explain when each variant is chosen.

Q2 Lowering of Data Structures [20 marks]

Marks: (a) 10 (b) 10

(a) **Arrays.** Write a C program declaring `int arr[5]`, writing values into it with a loop, and reading `arr[2]`. In the generated IR, address:

- How does the compiler represent array access using `getelementptr` (GEP)? Show one GEP instruction in the report, explaining each operand.
- How is the array itself allocated in IR? What does the `alloca` instruction look like for an array type, and how does it differ from a scalar `alloca`?
- Allocate one array statically and another one dynamically (using `malloc`), and identify the differences in the generated IR.

(b) **Structs and classes.** Write (i) a C program with `struct Point{ int x; int y;}` and (ii) a C++ program with an equivalent `class Point` having a constructor. In the IR for both:

- Locate the type definition and state how member alignment is encoded.
- Are there structural differences in the type layout between struct and class? How does the constructor appear in the class IR?
- How is accessing a member of a class or a struct different from accessing an element from an array?

Q3 LLVM IR Grammar and the Clang Frontend Pipeline [30 marks]

Marks: (a) 15 (b) 15

Use the following program as the single source for all three parts of this question:

```
int ternary_example(int a, int b) {
    return (a > b) ? (a * 2) : (b + 1);
}
```

(a) **Clang frontend pipeline: tokens → AST → IR.**
Clang exposes each stage of the frontend via dedicated flags. Run all three commands below on the program above and include the full output of each in your report.

i. **Lexer — token stream.** Dump every token produced by the lexer:

```
clang -cc1 -dump-tokens input.c 2>&1
```

For each distinct token *kind* that appears in the output (e.g. `int`, `identifier`, `greater`, `question`, `r_paren`, ...), state its category (keyword, operator, literal, punctuation, or identifier) and explain what information beyond the kind Clang records per token.

ii. **AST — abstract syntax tree.** Dump the Clang AST:

```
clang -Xclang -ast-dump -fsyntax-only input.c
```

In the output, locate and explain the following nodes:

- `FunctionDecl` — what information does it carry about `ternary_example`?
- `ParmVarDecl` — how are the parameters `a` and `b` recorded?
- `ConditionalOperator` — which child nodes correspond to the condition, the true branch, and the false branch?
- `BinaryOperator` — identify the node for `a > b` and state how the operator kind and operand types are represented.

Draw a **tree diagram** of the AST for the function body only, using the node names from the `--ast-dump` output as labels. You may draw this by hand (scan neatly) or use any tool.

(b) **Clang Optimization.** Generate the IR at `-O0` and then again at `-O1`:

```
clang -S -emit-llvm -O0 -o ternary_O0.ll input.c
clang -S -emit-llvm -O1 -o ternary_O1.ll input.c
```

Compare the two outputs and answer:

- How does the `ConditionalOperator` node in the AST map to IR instructions at `-O0`? Trace the path from AST node to IR.
- At `-O1`, does the ternary reduce to a `select` instruction? If so, explain what `select` does and why it is a valid replacement for the branching version.
- Provide five C program examples that can demonstrate the difference in `O0` and `O1` `ll` files and point out the name of the applied optimization(s) and explain your understanding of the optimization(s) for each example.

Deliverables

Submit a single ZIP archive named `<RollNumber>.zip` (e.g. `CS24BTECH12345.zip`) with the following structure:

```
CS24BTECH12345/
    report.pdf
    Q1/files
    Q2/files
    Q3/files
```

Report format. PDF only; no handwritten reports. One section per question. All IR snippets must use a monospace font — do not paste IR as screenshots. Annotate snippets directly to highlight the constructs being discussed. Maximum 20 pages, excluding source code appendices. State your OS and Clang version on page 1.

Policies

- **LLVM version.** All IR must be generated with Clang/LLVM 17.x.
- **Optimisation level.** Use `-O0` unless a sub-question explicitly states otherwise.
- **Platform.** Linux, macOS, or WSL2 are all acceptable.
- **Submission format.** Archives with incorrect directory structure or missing files incur a 10% penalty.
- **Late submissions.** No extensions will be granted. Submissions after the deadline receive a zero.
- Any usage of LLMs (ChatGPT, Claude, etc) must be clearly declared on what parts you used and how you have used. Please note that the report will be checked for originality and AI plagiarism. Copying from peers constitutes plagiarism and will result in an **FR grade**. The TAs may conduct oral examinations on any submitted work.

References

- LLVM Language Reference Manual: `https://llvm.org/docs/LangRef.html`
- LLVM Programmer's Manual: `https://llvm.org/docs/ProgrammersManual.html`
- Mapping High-Level Constructs to LLVM IR: `https://mapping-high-level-constructs-to-llvm-ir.readthedocs.io/`
- LLVM Tutorial — Kaleidoscope: `https://llvm.org/docs/tutorial/`